Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ

Гуманитарный институт
Кафедра информационных технологий в креативных и культурных индустриях

РЕФЕРАТ
по дисциплине «Информационные системы и технологии»
Технологии создания обработки текстовых документов

Преподаватель И. Р. Нигматуллин

Студент(ы) Бородина А. Б., Шеломенцева К. Д.
Группа: ГФ26-02Б

Красноярск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Основная часть 4
1.1 Кодирование текстовых данных 4
1.2 Проприетарные и открытые форматы текстовых документов 7
1.3 Языки разметки: разделение структуры и оформления документа 10
1.4 Метаданные и семантическая разметка текстов 13
1.5 Автоматическая обработка текста: методы и границы достоверности . 16
1.6 Долговременное хранение и архивация текстовых ресурсов 19
Заключение 22
Список сокращений 23
Список использованных источников 24

ВВЕДЕНИЕ

В условиях стремительного роста объёмов цифровой информации текстовые документы остаются одной из основных форм представления, хранения и передачи знаний в науке, управлении и производстве. Эффективность работы с ними зависит не только от удобства редакторов, но и от фундаментальных технологических решений: способов кодирования символов, выбора форматов файлов, использования языков разметки, применения метаданных и методов автоматической обработки. Ошибки на этих уровнях приводят к потере данных, несовместимости документов и невозможности их долговременного хранения.

Актуальность темы обусловлена переходом к многоязычным документам, распространением открытых стандартов, развитием методов автоматической обработки текста на основе искусственного интеллекта и необходимостью обеспечивать читаемость цифровых ресурсов на десятилетия вперёд. Понимание того, как устроены кодировки, форматы, разметка и стратегии архивации, становится базовым требованием для специалистов в области информационных систем и технологий.

Цель работы — исследовать технологии создания и обработки текстовых документов, включая эволюцию кодирования, форматы файлов, языки разметки, метаданные, методы автоматической обработки и стратегии долговременного хранения.

Для достижения цели необходимо решить следующие задачи:

– проанализировать эволюцию кодирования текстовых данных от национальных кодировок к Unicode;

– сопоставить проприетарные и открытые форматы текстовых документов с точки зрения совместимости и долговечности;

– рассмотреть языки разметки и принцип разделения структуры и оформления документа;

– охарактеризовать метаданные и семантическую разметку текстов;

– изучить методы автоматической обработки текста и границы достоверности их результатов;

– определить форматы и стратегии долговременного хранения и архивации текстовых ресурсов.

ОСНОВНАЯ ЧАСТЬ

1.1 Кодирование текстовых данных

Исторически первой формой представления текста в вычислительных системах стала кодовая страница — таблица соответствия между числовыми кодами и символами. В эпоху ранних ЭВМ, когда объёмы памяти измерялись килобайтами, а стандартизация только зарождалась, каждая страна и даже отдельные производители использовали собственные наборы символов. Это порождало проблему несовместимости: документ, созданный в одной системе, при переносе в другую отображался последовательностями нечитаемых символов. В основе большинства национальных кодировок лежит принцип однобайтового представления, при котором 256 возможных значений кода распределялись между управляющими символами, латиницей и национальным алфавитом. Для кириллицы классическими примерами являются КОИ-8 (разработанная для телетайпов и ЭВМ серии ЕС), Windows-1251 (получившая распространение с операционной системой Windows) и CP866 (использовавшаяся в DOS); для западноевропейских языков — ISO 8859-1 (Latin-1); для японского — Shift-JIS. При этом даже внутри одной языковой группы существовало несколько несовместимых вариантов: так, для русского языка одновременно применялись как минимум три кодировки, и выбор зависел от операционной системы, программного обеспечения и региона.

Фундаментальным ограничением национальных кодировок стала их неспособность представить многоязычный текст. В документе, содержащем одновременно русский, английский и, например, греческий текст, однобайтовая таблица не могла вместить все необходимые символы, что требовало переключения кодировок или применения «суррогатных» решений вроде комбинирования фрагментов с разными кодовыми страницами. Кроме того, кодировки не различали символ и его графическое начертание, не поддерживали диакритические знаки в полном объёме, не обеспечивали единой сортировки (например, правила упорядочивания букв «ё» и «е» различались) и не содержали символов для научной нотации, математических знаков и редких письменностей. Ещё одной проблемой была неоднозначность отображения: один и тот же код в разных кодировках соответствовал разным символам, что приводило к ошибкам при автоматической обработке и поиске.

Решение было предложено в конце 1980-х годов в рамках проекта Unicode, целью которого стало создание единой универсальной кодировки, охватывающей все письменные системы мира, включая исторические и искусственные [1]. Официальной датой рождения стандарта считается 1991 год, когда была опубликована первая версия Unicode 1.0. С тех пор стандарт постоянно развивается: в нём появляются новые блоки символов (эмодзи, редкие письменности, исторические алфавиты), а текущая версия содержит более 150 тысяч кодовых точек.

В основе Unicode лежит абстрактное понятие «кодовой точки» (code point) — числа в диапазоне от U+0000 до U+10FFFF, закреплённого за конкретным символом. Пространство кодовых точек организовано в 17 плоскостей (planes) по 65 536 символов в каждой. Нулевая плоскость (BMP, Basic Multilingual Plane) содержит наиболее употребительные символы, включая латиницу, кириллицу, греческий, арабский, иероглифы и большинство знаков пунктуации. Остальные плоскости отведены под дополнительные символы: эмодзи, редкие и исторические письменности, математические и музыкальные знаки. Символы сгруппированы в блоки, соответствующие письменностям (кириллица, латиница, арабский, хангыль и т. д.), что упрощает навигацию и обработку. Важно отметить, что Unicode кодирует именно символы, а не их начертания: один и тот же символ может отображаться разными шрифтами и глифами, а один глиф может представлять несколько символов (лигатуры).

Для практического представления кодовых точек в памяти и файлах используются различные схемы кодирования. UTF-8 — переменная длина от 1 до 4 байт, совместимая с ASCII: первые 128 символов кодируются одним байтом, что делает её оптимальной для англоязычных текстов и веб-документов. UTF-8 обладает свойством самосинхронизации: по первому байту можно определить длину последовательности, что упрощает восстановление после ошибок и поиск по тексту. Именно UTF-8 стала стандартом де-факто для веб-страниц, файлов конфигурации, исходного кода программ и большинства современных форматов обмена данными. UTF-16 использует 2 или 4 байта и применяется во внутренних представлениях многих операционных систем (Windows, Java, .NET) и языков программирования. Для символов за пределами BMP используются суррогатные пары (surrogate pairs) — комбинации двух 16-битных значений, что позволяет представить весь диапазон Unicode. UTF-32 фиксирует 4 байта на символ, обеспечивая простоту индексации, но увеличивая объём данных в 2–4 раза по сравнению с UTF-8 и UTF-16. Выбор схемы кодирования зависит от задачи: для хранения и передачи текста обычно предпочтительна UTF-8, для внутренней обработки в языках программирования — UTF-16 или UTF-32.

Помимо базовой кодировки, Unicode определяет механизмы нормализации (NFC, NFD, NFKC, NFKD), которые приводят составные символы к каноническому виду. Например, буква «é» может быть представлена как единый код (NFC) или как комбинация «e» и диакритического знака (NFD); без нормализации две визуально одинаковые строки могут считаться разными при сравнении и поиске. Нормализация критически важна для корректной работы систем аутентификации, поиска, сортировки и баз данных. С введением Unicode проблема «кракозябр» в основном решена, однако остаются практические сложности: не все шрифты содержат глифы для редких символов (что приводит к отображению «квадратиков»), а некоторые устаревшие системы и файлы по-прежнему используют национальные кодировки, что требует конвертации и может приводить к потере данных. Кроме того, существуют вопросы безопасности: визуально похожие символы из разных письменностей

(так называемые homoglyphs) могут использоваться для подделки доменных имён и фишинга.

Таким образом, эволюция кодирования текста прошла путь от локальных однобайтовых таблиц к глобальной многоуровневой системе Unicode, обеспечивающей многоязычность, расширяемость и совместимость. Понимание различий между кодовой точкой и схемой кодирования (UTF-8, UTF-16), а также механизмов нормализации является базовым требованием для корректной обработки текстовых данных в современных информационных системах.

1.2 Проприетарные и открытые форматы текстовых документов

Формат текстового документа определяет не только способ хранения символов и оформления, но и возможность его открытия, редактирования и долговременного сохранения. По этому признаку форматы делятся на проприетарные (закрытые) и открытые. Выбор формата — это всегда компромисс между функциональностью, совместимостью, свободой использования и долговечностью, и именно от него зависит, сможет ли документ быть прочитан через десять, двадцать или пятьдесят лет.

Проприетарные форматы контролируются конкретным производителем и часто описываются закрытой спецификацией, недоступной для независимой реализации. Классическим примером является формат DOC (Microsoft Word, начиная с 1980-х годов). Изначально Word для MS-DOS использовал простой текстовый формат, но с выходом Word for Windows в 1989 году формат стал бинарным контейнером со сложной внутренней структурой: заголовком, таблицей размещения блоков, потоками данных, встроенными объектами и служебной информацией. Эта структура менялась от версии к версии (Word 6.0, Word 95, Word 97–2003), и документ, созданный в новой версии, мог некорректно отображаться в старых версиях или в альтернативных офисных пакетах. Такая закрытость обеспечивала вендору конкурентное преимущество и позволяла наращивать функциональность, но создавала проблемы совместимости и вендорской зависимости: пользователи оказывались «заперты» в экосистеме одного производителя. Аналогичная ситуация характерна для формата RTF (Rich Text Format), который, будучи текстовым по синтаксису и изначально задуманным как формат обмена, также развивался проприетарно, имел множество несовместимых расширений и на практике не обеспечивал полной переносимости между приложениями. Другими примерами проприетарных форматов являются WPD (WordPerfect), Pages (Apple), XPS (Microsoft) и DVI (в ранних реализациях), каждый из которых имеет свои ограничения по совместимости и долговечности.

Помимо технической закрытости, проприетарные форматы создают юридические и экономические риски. Спецификация может быть недоступна для независимых разработчиков, что затрудняет создание альтернативных редакторов и инструментов обработки. Лицензионные ограничения могут запрещать обратную разработку или использование формата в конкурирующих продуктах. Кроме того, судьба формата зависит от судьбы производителя: если компания прекращает поддержку продукта или меняет политику, документы могут остаться без средства для открытия. История знает немало примеров, когда проприетарные форматы становились «мёртвыми» (например, форматы ранних текстовых процессоров WordStar и WordPerfect для DOS), и доступ к ним требовал эмуляции старых систем или специализированных конвертеров.

Открытые форматы специфицированы публично и могут быть реализованы любым разработчиком без лицензионных ограничений. Наиболее

значимым из них является ODF (OpenDocument Format), стандартизированный ISO/IEC 26300 в 2006 году и используемый в LibreOffice, OpenOffice, а также в ряде государственных систем (например, в некоторых странах ЕС и Латинской Америки он принят как обязательный формат для государственных документов). Документ ODF представляет собой ZIP-архив, содержащий XML-файлы с содержимым (content.xml), стилями (styles.xml), метаданными (meta.xml), настройками (settings.xml) и вложениями. Это обеспечивает прозрачность структуры, возможность машинной обработки и независимость от конкретного вендора [2]. ODF существует в нескольких версиях (1.0, 1.1, 1.2, 1.3) и профилях, включая ODF для офисных приложений и ODF для архивов.

Ответом Microsoft на ODF стал формат OOXML (Office Open XML), также основанный на ZIP-контейнере и XML-разметке, но с иной схемой. OOXML был стандартизирован как ISO/IEC 29500 в 2008 году, однако его спецификация значительно объёмнее и сложнее, чем у ODF (тысячи страниц против сотен), а реализация в Word содержит расширения, отсутствующие в стандарте. В результате документы DOCX, будучи формально открытыми, на практике могут содержать элементы, корректно интерпретируемые только продуктами Microsoft [3]. OOXML существует в двух вариантах: Transitional (с поддержкой устаревших функций) и Strict (более строгий, но менее распространённый). Критики OOXML указывали на сложность спецификации, наличие дублирующих способов описания одних и тех же элементов и трудности независимой реализации, что стало предметом длительных дискуссий в сообществе и при принятии стандарта ISO.

Отдельного внимания заслуживает PDF (Portable Document Format) — формат фиксированной вёрстки, разработанный Adobe в 1993 году и изначально проприетарный, но с 2008 года открытый (ISO 32000). PDF не предназначен для редактирования, а фиксирует внешний вид документа, что делает его идеальным для обмена и архивирования, но неудобным для совместной работы. PDF поддерживает встраивание шрифтов, векторную графику, сжатие изображений, шифрование и цифровые подписи. Для долговременного хранения документов используется PDF/A — профиль, стандартизированный ISO 19005, который запрещает внешние ссылки, шифрование и встроенные шрифты без встраивания, что гарантирует воспроизводимость в будущем. PDF/A существует в нескольких версиях (PDF/A-1, A-2, A-3, A-4), различающихся уровнем поддержки функций и требований к метаданным.

Помимо перечисленных, существуют и другие открытые форматы, ориентированные на разные задачи. TXT (plain text) — простейший формат, содержащий только символы без оформления; он максимально долговечен, но не поддерживает форматирование. HTML и Markdown используются для вебдокументов и документации, но не всегда подходят для печати и сложной вёрстки. EPUB — открытый формат для электронных книг, основанный на XHTML и CSS. ODT и DOCX — основные форматы редактируемых документов, а PDF/A и ODF — предпочтительные форматы для архивирования.

Проблема совместимости проявляется на нескольких уровнях. Синтаксический уровень — корректность разбора файла: даже если формат открыт, разные реализации могут по-разному интерпретировать неоднозначные места спецификации. Семантический уровень — одинаковое понимание стилей и разметки: например, один и тот же стиль «Заголовок 1» может иметь разные параметры в разных приложениях. Визуальный уровень — одинаковое отображение: различия в шрифтах, алгоритмах вёрстки и поддержке расширений приводят к тому, что документ выглядит по-разному в разных программах. Даже формально открытые форматы не гарантируют полной идентичности рендеринга, что особенно критично для юридических и финансовых документов.

Долговечность формата, в свою очередь, зависит от нескольких факторов: распространённости (чем больше пользователей и приложений поддерживают формат, тем выше шансы на его сохранение), простоты спецификации (простые форматы легче реализовать независимо), наличия независимых реализаций (если формат поддерживается только одним производителем, риск утраты выше) и открытости (публичная спецификация снижает риск вендорской зависимости). Практика показывает, что для долговременного архивирования предпочтительны простые, широко документированные форматы (PDF/A, ODF, plain text), а для активной работы — форматы, поддерживаемые используемым программным обеспечением. Многие организации применяют стратегию «двух форматов»: рабочий формат для редактирования и архивный формат для долговременного хранения.

Таким образом, выбор формата текстового документа представляет собой компромисс между функциональностью, совместимостью, свободой использования и долговечностью. Открытые стандарты снижают риск вендорской зависимости, однако требуют внимательного отношения к профилям и расширениям, а проприетарные форматы обеспечивают богатую функциональность ценой потенциальной несовместимости. Понимание этих компромиссов является необходимым условием для грамотного проектирования систем документооборота и архивов.

1.3 Языки разметки: разделение структуры и оформления документа

Язык разметки — это система правил, определяющая, каким образом текст и другие элементы документа должны быть размечены для последующей обработки, отображения или хранения. Разметка позволяет отделить содержание документа от его представления и указать, какую роль играет тот или иной фрагмент: является ли он заголовком, абзацем, списком, таблицей, ссылкой или формулой. Ключевая идея современных языков разметки — разделение структуры (что представляет собой элемент) и оформления (как этот элемент выглядит: шрифт, цвет, отступы). Такое разделение позволяет использовать один и тот же контент в разных контекстах, автоматизировать обработку и обеспечивать единообразие оформления [4].

Исторически первым широко распространённым языком разметки стал SGML (Standard Generalized Markup Language), стандартизированный ISO в 1986 году. SGML не задаёт конкретный набор тегов, а определяет правила их описания через DTD (Document Type Definition), что делает его метаязыком — языком для создания других языков разметки. SGML использовался в крупных издательских и государственных системах (например, в документации Министерства обороны США), но был сложен для освоения и требовал специализированного программного обеспечения. На основе SGML были созданы HTML (для веб-документов) и XML (для структурированных данных).

HTML (HyperText Markup Language) был разработан Тимом Бернерсом-Ли в начале 1990-х годов для описания веб-страниц и стал основой Всемирной паутины. HTML фиксирует набор тегов для описания структуры документа (h1, p, ul, table), однако в ранних версиях (HTML 3.2, HTML 4.0) он смешивал структуру и оформление: теги font, b, i, center напрямую задавали внешний вид, что затрудняло поддержку, автоматическую обработку и адаптацию к разным устройствам. С появлением CSS (Cascading Style Sheets) в конце 1990-х годов оформление было вынесено в отдельные таблицы стилей, а HTML вернулся к описанию семантической структуры. Современный стандарт HTML5 (2014) закрепил семантические теги (header, nav, article, section, footer), которые описывают роль элемента, а не его внешний вид. CSS, в свою очередь, позволяет задавать стили для разных устройств (экран, печать, мобильные устройства), что обеспечивает адаптивность и единообразие оформления.

XML (Extensible Markup Language) был разработан в 1996–1998 годах как упрощённая версия SGML для обмена данными в веб-среде. XML не имеет фиксированного набора тегов, а позволяет определять собственные схемы (XSD, DTD), что делает его пригодным для обмена данными между системами, конфигураций, документов и т. д. На основе XML построены многие современные форматы: OOXML, ODF, SVG (векторная графика), RSS (новостные ленты), SOAP (веб-сервисы), XHTML. XML обладает строгим синтаксисом (все теги должны быть закрыты, атрибуты — в кавычках, документ должен иметь один корневой элемент), что обеспечивает однозначность разбора. Однако XML избыточен

для простых задач и требует значительных объёмов служебной информации: даже небольшой документ может содержать десятки строк разметки. Кроме того, XML не задаёт семантику тегов — и могут означать что угодно, если не определена схема.

Markdown — облегчённый язык разметки, предложенный Джоном Грубером в 2004 году. Он использует простой синтаксис на основе символов: решётки для заголовков, звёздочки для выделения (курсив, жирный), дефисы для списков, квадратные скобки для ссылок. Markdown легко читается человеком и без специальной обработки, что делает его удобным для написания документации, ведения заметок и работы в системах контроля версий. Markdown получил широкое распространение в блогах, документации (GitHub, GitLab), инструментах создания заметок (Obsidian, Notion) и статических генераторах сайтов (Jekyll, Hugo). Его ограничение — отсутствие строгой спецификации: существует множество диалектов (CommonMark, GitHub Flavored Markdown, MultiMarkdown), которые добавляют свои расширения (таблицы, сноски, математические формулы), что может приводить к различиям в отображении. Для решения этой проблемы в 2014 году был предложен стандарт CommonMark, который описывает базовый синтаксис и семантику.

LaTeX — язык разметки и система вёрстки, разработанная Лесли Лэмпортом в 1984 году на основе системы TeX Дональда Кнута. В отличие от WYSIWYG-редакторов, LaTeX использует команды для описания структуры документа (section, subsection, begin{equation}, cite), а оформление определяется стилевыми пакетами. Это обеспечивает высокое качество вёрстки, особенно для формул, таблиц и ссылок, и воспроизводимость результатов: один и тот же исходный файл компилируется в одинаковый PDF на разных системах. LaTeX широко используется в научных и технических публикациях, диссертациях и книгах. Его недостатки — сложность освоения, необходимость компиляции и ограниченная поддержка визуального редактирования. Typst — более современный аналог, разработанный в 2023 году, который сочетает простоту синтаксиса с высокой скоростью компиляции и возможностью программирования. Typst использует функциональный подход к вёрстке, поддерживает модули и пакеты, а также инкрементальную компиляцию, что делает его привлекательным для создания документов, требующих автоматизации [5].

Помимо перечисленных, существуют и другие языки разметки, ориентированные на конкретные задачи: YAML и JSON — для конфигураций и обмена данными; SVG — для векторной графики; MathML — для математических формул; DocBook — для технической документации; AsciiDoc — как альтернатива Markdown с более богатыми возможностями. Каждый из них имеет свои сильные и слабые стороны, и выбор зависит от задачи, аудитории и инструментов.

Общая тенденция развития языков разметки — движение от смешения структуры и оформления к их строгому разделению. Это позволяет отделить

содержание документа от его представления, облегчает автоматическую обработку, повторное использование и адаптацию к разным устройствам (веб, печать, мобильные экраны). Разделение структуры и оформления лежит в основе современных систем управления контентом (CMS), компонентного подхода в веб-разработке и принципов доступности (accessibility), когда один и тот же контент может быть представлен разным пользователям в разной форме. Понимание принципов разметки является необходимым условием для работы с современными информационными системами, системами управления контентом и инструментами автоматизации документооборота.

Таким образом, языки разметки прошли путь от сложных метаязыков (SGML) через веб-стандарты (HTML, XML) к облегчённым и специализированным решениям (Markdown, LaTeX, Typst). Ключевым принципом остаётся разделение структуры и оформления, которое обеспечивает гибкость, совместимость и долговечность документов.

1.4 Метаданные и семантическая разметка текстов

Метаданные — это данные о данных, то есть структурированная информация, описывающая содержание, контекст, происхождение, формат и другие характеристики документа. В отличие от самого текста, метаданные не отображаются непосредственно при чтении, но играют ключевую роль в поиске, классификации, управлении правами доступа и долговременном хранении. Без метаданных документ остаётся изолированным фрагментом текста, тогда как с ними он превращается в полноправный элемент информационной системы, который можно находить, связывать с другими ресурсами и обрабатывать автоматически.

Метаданные текстового документа можно условно разделить на несколько категорий, которые взаимодополняют друг друга. Описательные метаданные включают автора, название, дату создания, аннотацию, ключевые слова, язык и предметную область — они используются для каталогизации и поиска. Административные метаданные содержат сведения о правах доступа, лицензии, версии, сроках хранения, ответственном лице и истории изменений — они необходимы для управления документом на протяжении его жизненного цикла. Структурные метаданные описывают внутреннюю организацию документа: разделы, главы, таблицы, рисунки, связи между ними, оглавление; они позволяют навигировать по документу и извлекать его части. Технические метаданные фиксируют формат файла, кодировку, размер, использованные шрифты, программное обеспечение, контрольные суммы; они важны для воспроизводимости и проверки целостности. Отдельно выделяют статистические метаданные (число страниц, слов, символов) и правовые метаданные (условия использования, ограничения на копирование и распространение).

Исторически метаданные в текстовых документах присутствовали в виде служебных полей и колонтитулов, но их структурированное использование началось с появлением стандартов описания ресурсов. Одним из первых и наиболее универсальных стандартов стал Dublin Core (1995), который определил 15 базовых элементов для описания любых цифровых ресурсов: Title, Creator, Subject, Description, Publisher, Contributor, Date, Type, Format, Identifier, Source, Language, Relation, Coverage, Rights. Dublin Core используется в библиотеках, репозиториях и системах управления документами, а его элементы часто встраиваются в HTML-страницы через тег . Со временем Dublin Core был расширен (DCMI Metadata Terms) и дополнен другими схемами, но остаётся базовым стандартом для описательных метаданных.

В современных офисных форматах метаданные хранятся в специальных разделах. В OOXML это файлы core.xml (основные свойства: автор, название, дата, ключевые слова) и app.xml (сведения о приложении: версия, компания, время редактирования) внутри ZIP-контейнера. В ODF — meta.xml, который использует собственный набор элементов, совместимый с Dublin Core. В PDF метаданные хранятся в словаре Info (простые пары «ключ — значение») и

в XMP-пакетах. Стандарт XMP (Extensible Metadata Platform), разработанный Adobe в 2001 году, позволяет встраивать метаданные в различные форматы (PDF, JPEG, TIFF, PNG) в виде RDF/XML-описаний, что обеспечивает совместимость между приложениями и возможность автоматической обработки [6]. XMP поддерживает как стандартные схемы (Dublin Core, Photoshop, Camera Raw), так и пользовательские расширения, что делает его гибким инструментом для описания документов, изображений и медиафайлов.

Семантическая разметка идёт дальше структурной и стремится выразить смысл элементов документа, а не только их роль в вёрстке. В веб-контексте для этого используется Microdata, RDFa и JSON-LD, позволяющие указать поисковым системам, что перед ними рецепт, статья, событие, товар или организация. Стандарт Schema.org, созданный в 2011 году совместно Google, Microsoft, Yahoo и Yandex, определяет словарь типов и свойств для описания сущностей. Например, разметка статьи может содержать поля headline, author, datePublished, articleBody, image; разметка рецепта — recipeIngredient, cookTime, nutrition; разметка события — startDate, location, performer. Такая разметка улучшает индексирование, позволяет поисковым системам формировать расширенные сниппеты (rich snippets) и повышает видимость контента. JSON-LD, в отличие от Microdata и RDFa, выносит разметку в отдельный блок, что упрощает её генерацию и поддержку.

В области цифровых гуманитарных наук и архивного дела применяется стандарт TEI (Text Encoding Initiative), разработанный в 1987 году и с тех пор постоянно развивающийся. TEI — это схема XML для представления текстов с полной семантической разметкой: структурой (разделы, абзацы, стихи, драматические реплики), вариантами (рукописи, редакции), примечаниями, ссылками, персоналиями, географическими названиями, датами. TEI позволяет создавать машиночитаемые научные издания, пригодные для анализа, визуализации и долговременного хранения [7]. Крупнейшие цифровые библиотеки и проекты (Perseus, TextGrid, Europeana) используют TEI для описания текстов, что обеспечивает совместимость и возможность обмена данными между исследовательскими группами.

Помимо TEI, существуют и другие схемы семантической разметки: EAD (Encoded Archival Description) — для описания архивных фондов; METS (Metadata Encoding and Transmission Standard) — для упаковки метаданных и цифровых объектов; MODS (Metadata Object Description Schema) — для библиографических описаний; PREMIS — для метаданных сохранности. Эти стандарты образуют экосистему, в которой каждый отвечает за свой аспект описания и хранения цифровых ресурсов.

Семантическая разметка открывает возможности для автоматической обработки текста: извлечения сущностей (имена, организации, даты, места), построения связей между документами, тематического моделирования, поддержки поиска по смыслу, а не только по ключевым словам. Например, систе-

ма, размечающая упоминания персон и организаций в новостных статьях, может автоматически строить граф связей и выявлять ключевых участников событий. Однако её эффективность зависит от качества и полноты разметки, а также от согласованности используемых словарей: если разные документы размечены по-разному, автоматическая обработка может давать противоречивые результаты. В корпоративной практике семантическая разметка часто ограничивается метаданными документов и разметкой веб-страниц, тогда как глубокая семантическая аннотация (например, разметка всех упоминаний сущностей в тексте) остаётся трудоёмкой и требует участия экспертов или применения методов машинного обучения.

Отдельно стоит упомянуть проблему совместимости метаданных. Разные системы используют разные схемы, разные имена полей и разные форматы значений (например, дата может быть записана как 2026-11-12, 12.11.2026 или November 12, 2026). Для решения этой проблемы применяются кросс-схемы (crosswalks) — таблицы соответствия между элементами разных схем, а также общие словари и онтологии (например, FOAF для описания людей, SKOS для тезаурусов). Европейская инициатива Europeana разработала модель данных, которая агрегирует метаданные из разных источников и приводит их к единому виду, что позволяет искать объекты культурного наследия по всей Европе.

Таким образом, метаданные и семантическая разметка превращают текст из простой последовательности символов в структурированный информационный объект, пригодный для автоматизированного поиска, анализа и управления. Их роль возрастает по мере роста объёмов текстовых ресурсов и развития технологий обработки естественного языка.

1.5 Автоматическая обработка текста: методы и границы достоверности

Автоматическая обработка текста (Natural Language Processing, NLP) — область информатики и искусственного интеллекта, изучающая методы анализа и преобразования текстов на естественном языке с помощью компьютерных программ. Задачи NLP разнообразны: от простых операций (поиск, замена, подсчёт частот) до сложных (машинный перевод, извлечение знаний, генерация текста, ответы на вопросы). В отличие от обработки структурированных данных, где правила однозначны, работа с естественным языком сопряжена с многозначностью, неполнотой и вариативностью, что делает её принципиально сложной задачей.

Методы обработки текста можно разделить на несколько групп, которые исторически сменяли друг друга, но до сих пор применяются совместно. Символьные и лексические методы включают токенизацию (разбиение текста на слова и знаки), нормализацию (приведение к нижнему регистру, удаление пунктуации), стемминг (отсечение окончаний, например, «бегущий» → «бег») и лемматизацию (приведение к словарной форме: «бегущий» → «бежать»). Эти методы служат основой для большинства последующих алгоритмов. Синтаксические методы анализируют структуру предложений: части речи (POS-tagging), зависимости между словами (dependency parsing), деревья разбора (constituency parsing). Семантические методы стремятся извлечь смысл: именованные сущности (NER — имена, организации, даты, места), связи между ними (relation extraction), тональность (sentiment analysis), темы (topic modeling), семантические роли. Статистические и вероятностные методы (модели n-грамм, наивный байесовский классификатор, скрытые марковские модели, латентное размещение Дирихле) долгое время были основой NLP и до сих пор используются в задачах с ограниченными данными.

С начала 2010-х годов доминирующее положение заняли нейронные сети. Рекуррентные сети (RNN, LSTM, GRU) хорошо работали с последовательностями, но страдали от затухания градиентов на длинных текстах. Прорыв произошёл в 2017 году с появлением архитектуры трансформеров (Transformer), которая позволила эффективно обрабатывать длинные зависимости и распараллеливать вычисления. На основе трансформеров были созданы модели BERT (2018), GPT (2018–2023), T5, RoBERTa, LLaMA и другие, которые обеспечивают высокое качество на многих задачах: машинный перевод, суммаризация, ответы на вопросы, генерация текста [8]. Современные большие языковые модели (Large Language Models, LLM) содержат миллиарды параметров и обучаются на огромных корпусах текстов, что позволяет им решать задачи, которые ранее требовали специализированных моделей.

Практические применения автоматической обработки текста включают: поиск и фильтрацию информации (поисковые системы, рекомендательные системы), классификацию документов (рубрикация новостей, сортировка обращений), извлечение данных из неструктурированных текстов (например,

извлечение реквизитов из договоров), машинный перевод, суммаризацию (краткое изложение документов), проверку орфографии и грамматики, анализ тональности (мониторинг отзывов и социальных сетей), чат-боты и голосовые ассистенты, генеративные модели (написание текстов, создание кода). В информационных системах NLP используется для автоматической рубрикации, интеллектуального поиска, обработки обращений пользователей и поддержки принятия решений.

Однако достоверность результатов автоматической обработки имеет границы, и их важно осознавать. Во-первых, естественный язык многозначен: одно и то же слово может иметь разные значения в зависимости от контекста («ключ» — инструмент, источник воды, музыкальный знак), а ирония, сарказм и намёки плохо поддаются формализации. Во-вторых, качество моделей зависит от обучающих данных: если корпус нерепрезентативен (например, содержит только формальные тексты, но не разговорную речь), модель будет систематически ошибаться на новых текстах. Это явление называется смещением данных (data bias). В-третьих, нейронные модели часто работают как «чёрный ящик»: даже если модель выдаёт правильный ответ, объяснить, как она к нему пришла, бывает сложно, что затрудняет контроль ошибок и доверие к результатам. В-четвёртых, существуют этические и правовые ограничения: обработка персональных данных, авторские права (обучение моделей на защищённых текстах), генерация недостоверной информации (галлюцинации языковых моделей, deepfake-тексты), возможность манипуляции общественным мнением.

Для повышения достоверности применяются несколько подходов. Гибридные методы сочетают правила и машинное обучение: например, для извлечения дат из документов можно использовать регулярные выражения вместе с нейросетевой моделью. Ансамбли моделей объединяют результаты нескольких алгоритмов для снижения ошибок. Активное обучение предполагает участие эксперта, который проверяет и корректирует результаты модели, а затем эти исправления используются для дообучения. Валидация на независимых выборках позволяет оценить, насколько модель обобщает знания, а не запоминает обучающие примеры. Интерпретируемость (explainable AI) помогает понять, какие признаки влияли на решение модели, и выявить систематические ошибки. Важно понимать, что автоматическая обработка текста не заменяет эксперта полностью, а выступает инструментом поддержки, требующим верификации результатов в критических областях (медицина, право, финансы).

Отдельно стоит упомянуть проблему оценки качества. Для разных задач используются разные метрики: точность (accuracy), полнота (recall), F1-мера для классификации; BLEU, ROUGE, METEOR для машинного перевода и суммаризации; perplexity для языковых моделей. Однако метрики не всегда отражают реальное качество: модель может иметь высокий BLEU, но выдавать грамматически корректные, но бессмысленные переводы. Поэтому всё

большее значение приобретает оценка человеком (human evaluation) и тестирование на реальных сценариях.

Таким образом, автоматическая обработка текста прошла путь от простых эвристик к мощным нейросетевым моделям, однако её применение требует осознания ограничений и ответственности за результаты. Научно обоснованная оценка достоверности и прозрачность методов остаются ключевыми условиями эффективного использования NLP в информационных системах.

1.6 Долговременное хранение и архивация текстовых ресурсов

Долговременное хранение текстовых ресурсов — задача, выходящая за рамки простого сохранения файлов. Она включает обеспечение читаемости, целостности, аутентичности и доступности документов на протяжении десятилетий, а иногда и столетий, при смене технологий, форматов, носителей и организаций. В отличие от оперативного хранения, где достаточно резервного копирования, долговременное хранение требует стратегического планирования и учёта рисков, связанных с устареванием технологий. Цифровые документы, в отличие от бумажных, не могут «лежать на полке» без внимания: без активных действий они становятся недоступными гораздо быстрее, чем бумага (которая может храниться веками, хотя и требует условий).

Основные угрозы долговременному хранению можно разделить на технологические и организационные. Технологические угрозы включают устаревание носителей (магнитные ленты, дискеты, оптические диски CD/DVD, ранние жёсткие диски), устаревание форматов (невозможность открыть файл в современном ПО), устаревание программного обеспечения (прекращение поддержки редакторов, операционных систем), потерю целостности из-за сбоев, битовых ошибок (bit rot) и деградации носителей. Организационные угрозы включают недостаточное финансирование, утрату документации, смену персонала (уход специалистов, знающих систему), правовые ограничения (например, лицензии на ПО, необходимые для чтения формата), ошибки каталогизации и недостаток компетенций. Отдельно стоит упомянуть угрозы безопасности: несанкционированный доступ, кибератаки, шифрование-вымогательство и случайное удаление.

Стратегии долговременного хранения включают несколько уровней, которые обычно применяются совместно. Выбор устойчивых форматов — предпочтение отдаётся открытым, широко документированным и простым форматам: plain text (UTF-8) для текста без форматирования, PDF/A для документов с фиксированной вёрсткой, ODF для редактируемых документов, TIFF для изображений, WAV для аудио. Чем проще формат, тем легче его реализовать независимо и тем выше шансы на долговременную поддержку. Миграция — периодическое преобразование документов в актуальные форматы с проверкой целостности и сохранением оригиналов. Миграция может быть «миграцией на носитель» (перенос данных с устаревающих носителей на новые) и «миграцией формата» (конвертация в новый формат). Эмуляция — сохранение оригинального программного обеспечения и среды (например, виртуальной машины) для воспроизведения старых форматов. Эмуляция позволяет сохранить «аутентичный вид» документа, но требует поддержки эмуляторов и лицензий на старое ПО. Инкапсуляция — хранение документа вместе с описанием формата и необходимым ПО (например, в контейнерах), что позволяет воспроизвести его даже без внешних ресурсов. Репликация — создание нескольких копий в разных местах и на разных носителях (стратегия

LOCKSS — Lots of Copies Keep Stuff Safe), что снижает риск потери из-за локальных катастроф [9]. Современные подходы также включают географическое распределение (копии в разных странах и дата-центрах) и облачное хранение с проверкой целостности.

Стандарты и модели долговременного хранения играют ключевую роль в обеспечении совместимости и воспроизводимости. Модель OAIS (Open Archival Information System, ISO 14721:2012) описывает функциональные компоненты архива: приём (Ingest), архивное хранение (Archival Storage), управление данными (Data Management), администрирование (Administration), планирование сохранности (Preservation Planning) и доступ (Access). OAIS также определяет информационные пакеты: SIP (Submission Information Package) — пакет, поступающий в архив; AIP (Archival Information Package) — пакет, хранящийся в архиве; DIP (Dissemination Information Package) — пакет, выдаваемый пользователю. Эта модель лежит в основе многих национальных и корпоративных архивов, включая Национальные архивы США (NARA), Европейский архив (European Archives) и российские государственные архивы. Стандарт PDF/A (ISO 19005) определяет профиль PDF, гарантирующий воспроизводимость: запрет внешних ссылок, встраивание шрифтов, использование стандартизированных цветовых пространств, запрет шифрования (в PDF/A-1 и A-2). PDF/A существует в нескольких версиях: A-1 (2005), A-2 (2011), A-3 (2012), A-4 (2020), различающихся уровнем поддержки функций. Стандарт ISO 15489 описывает управление документами (records management), включая политику, процедуры и требования к метаданным. Для научных данных используется стандарт OAIS и модель FAIR (Findable, Accessible, Interoperable, Reusable), которая задаёт требования к метаданным и форматам, облегчающие повторное использование.

Практика цифровых архивов (национальные библиотеки, государственные архивы, научные репозитории) показывает, что устойчивое хранение требует сочетания технических, организационных и правовых мер. Технические меры включают регулярное резервное копирование с проверкой восстановления, мониторинг целостности (контрольные суммы, хеши, проверка фиксации ошибок), использование отказоустойчивых систем хранения (RAID, распределённые файловые системы), а также автоматизацию процессов миграции и проверки. Организационные меры включают документирование форматов и процедур, обучение персонала, создание политик хранения и регулярный аудит. Правовые меры включают юридическое закрепление ответственности за архив, соблюдение требований к персональным данным и авторским правам, а также лицензирование ПО, необходимого для чтения форматов.

Отдельного внимания заслуживает проблема аутентичности. Документ, хранящийся десятилетиями, должен оставаться тем же документом, что был создан: его содержание, структура и метаданные не должны изменяться. Для подтверждения аутентичности используются цифровые подписи, хеширование, журналирование и контроль версий. Однако цифровые подписи имеют

срок действия и могут перестать быть валидными через несколько лет, поэтому для долговременного хранения требуются механизмы долговременной валидации (long-term validation), такие как метки времени (timestamping) и архивирование сертификатов.

Таким образом, долговременное хранение текстовых ресурсов — комплексная проблема, решаемая не выбором одного «вечного» формата, а системой взаимодополняющих стратегий: устойчивые форматы, миграция, репликация, метаданные, стандарты и организационные регламенты. Осознанное проектирование архива на ранних этапах жизненного цикла документа значительно снижает риск утраты информации в будущем.

ЗАКЛЮЧЕНИЕ

В ходе работы были рассмотрены теоретические основы информационных технологий и систем, а также технологии создания и обработки текстовых документов. Установлено, что ИТ и ИС не тождественны: ИС задаёт организационно-технический каркас, а ИТ наполняет его методами и алгоритмами обработки данных. Рассмотрены классификация систем, этапы их жизненного цикла и модели разработки, а также назначение АИС и АСУ, включая роль обратной связи в управлении.

В части обработки текстов показано, что переход от национальных кодировок к Unicode решил проблему многоязычности и совместимости. Сравнение проприетарных и открытых форматов выявило компромисс между функциональностью и долговечностью, а разделение структуры и оформления в языках разметки обеспечило гибкость автоматизированной обработки. Рассмотрены метаданные, методы автоматической обработки текста с их ограничениями, а также стратегии долговременного хранения, основанные на устойчивых форматах, миграции и стандартах.

Таким образом, цель работы достигнута: системный и документный аспекты обработки информации взаимодополняют друг друга, а их грамотная организация является условием эффективного функционирования современных информационных систем.

СПИСОК СОКРАЩЕНИЙ

AIP — Archival Information Package (архивный информационный пакет)
BMP — Basic Multilingual Plane (основная многоязычная плоскость Unicode)
BOM — Byte Order Mark (маркер порядка байтов)
CMS — Content Management System (система управления контентом)
CSS — Cascading Style Sheets (каскадные таблицы стилей)
DIP — Dissemination Information Package (пакет выдачи информации)
DTD — Document Type Definition (определение типа документа)
EAD — Encoded Archival Description (кодированное архивное описание)
FAIR — Findable, Accessible, Interoperable, Reusable (находимый, доступный, совместимый, повторно используемый)
HTML — HyperText Markup Language (язык гипертекстовой разметки)
JSON — JavaScript Object Notation (текстовый формат обмена данными)
JSON-LD — JavaScript Object Notation for Linked Data (JSON для связанных данных)
LLM — Large Language Model (большая языковая модель)
LOCKSS — Lots of Copies Keep Stuff Safe (много копий сохраняют данные)
METS — Metadata Encoding and Transmission Standard (стандарт кодирования и передачи метаданных)
MODS — Metadata Object Description Schema (схема описания метаданных объекта)
NLP — Natural Language Processing (обработка естественного языка)
OAIS — Open Archival Information System (открытая архивная информационная система)
ODF — OpenDocument Format (открытый формат документов)
OOXML — Office Open XML (открытый формат офисных документов)
PDF — Portable Document Format (портативный формат документов)
PDF/A — PDF for Archiving (профиль PDF для долговременного архивирования)
PREMIS — Preservation Metadata: Implementation Strategies (метаданные сохранности)
RDF — Resource Description Framework (фреймворк описания ресурсов)
RTF — Rich Text Format (формат расширенного текста)
SGML — Standard Generalized Markup Language (стандартный обобщённый язык разметки)
SIP — Submission Information Package (пакет передачи информации)
TEI — Text Encoding Initiative (инициатива кодирования текстов)
UTF — Unicode Transformation Format (формат преобразования Unicode)
XMP — Extensible Metadata Platform (расширяемая платформа метаданных)
XML — Extensible Markup Language (расширяемый язык разметки)
XSD — XML Schema Definition (определение схемы XML)
XSLT — Extensible Stylesheet Language Transformations (преобразования расширяемого языка таблиц стилей)

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The Unicode Standard : official site / Unicode Consortium. — URL: https://www.unicode.org/standard/standard.html (дата обращения: 12.11.2026).
2. OASIS OpenDocument Format for Office Applications (OpenDocument) Version 1.3. — URL: https://docs.oasis-open.org/office/OpenDocument/v1.3/ (дата обращения: 12.11.2026).
3. ISO/IEC 29500-1:2016. Information technology — Document description and processing languages — Office Open XML File Formats. — URL: https://www.iso.org/standard/71691.html (дата обращения: 12.11.2026).
4. W3C. HTML & CSS : official site. — URL: https://www.w3.org/standards/webdesign/htmlcss (дата обращения: 12.11.2026).
5. Typst Documentation. — URL: https://typst.app/docs/ (дата обращения: 12.11.2026).
6. Adobe XMP Specification. — URL: https://www.adobe.com/devnet/xmp.html (дата обращения: 12.11.2026).
7. TEI: Text Encoding Initiative. — URL: https://tei-c.org/ (дата обращения: 12.11.2026).
8. Stanford NLP Group : official site. — URL: https://nlp.stanford.edu/ (дата обращения: 12.11.2026).
9. Digital Preservation Coalition : official site. — URL: https://www.dpconline.org/ (дата обращения: 12.11.2026).